Databases, SQL, Pipelines & Data Storage for the AI/ML Era

Every dataset students have ever touched in COMET, prAxIs or in-class was the final artifact of a pipeline someone else built. Someone designed the tables, wrote the queries, scheduled the jobs, and chose the storage format. This stream teaches what happens before the CSV, because that is where most real data and AI work actually happens.

Pitch: “In COMET you learned what to do once you have a dataframe. But in industry and in modern research, nobody hands you a ready dataframe. Data lives in databases, arrives continuously, and has to be extracted, shaped, validated, and kept flowing. LLMs are trained on pipelines that crawl and filter the web, RAG systems (a database attached to a model). This stream teaches databases, SQL, and pipelines as the foundation layer of AI/ML and shows why LLMs being able to write SQL for you makes these skills more valuable, not less: the job shifts from typing queries to designing schemas and verifying correctness, and you cannot verify what you don't understand."

Positioning/Alternatives:

- **CPSC 368 (Databases in Data Science):** closest existing course, explicitly aimed at data science use of databases gated behind CPSC 203/210/CPEN 221/DSCI221, so Econ/Arts students don’t really take it. This stream covers its most employable core (DBMS concepts, ER modeling, SQL, OLAP/warehousing, NoSQL) at an applied level, minus the CS-internals material (B+-trees, hashing, ARIES).
- **CPSC 304:** database *design/theory* course, same prereq wall, Java/PHP oriented.
- **Masters in Data Science / School of Information grad courses:** graduate admission required.
- **Nothing in the Econ/Arts curriculum** teaches SQL (that I know of), despite it being one of the most frequently listed skills in data-analyst and data-science job postings.

The stream will have a common dataset example (like the Causal ML stream had) throughout all eleven notebooks. Or if it's hard to fit 2-3 common datasets which repeat and are re-introduced.

We also end each notebook with a short "you can now answer these interview questions" box (what's a foreign key; how does a left join change row counts; what's idempotency; OLTP vs OLAP) and point students at free SQL practice grinders (DataLemur, StrataScratch, LeetCode SQL) after NB2.

Each notebook also has a you are here on a mermaid diagram of the stream.

Notebook 1: Why do Databases exist and what are they?

- We begin like any COMET notebook with loading a dataframe (this time on purpose it is in a horrible state, inconsistent, overlapping, duplicates, multiple entries, corrections, columns change meanings etc). We try to answer a simple question of getting an average value and we fail. The output is incorrect and messy.
- Normal files vs Database, define key concepts like database and SQL

- Why you need a Database Management System (DBMS).
- We load different clean data into SQLite, run commands like SELECT, WHERE, ORDER BY, and it all is far easier. First introduction to SQL.
- Roadmap of the whole stream lives here (simplified version of this outline)

Goal: students state why flat files fail (redundancy, anomalies, concurrency), explain what a DBMS provides, run basic single-table SQL queries in a notebook.

Notebook 2: Basic Querying of Real Data

- The core of SQL on real econ data: everything in this notebook is a single SELECT
- SELECT, WHERE, ORDER BY, LIMIT (quick expansion of NB1), DISTINCT.
- JOIN (inner/left) and why row counts change
- GROUP BY / HAVING and aggregates (COUNT, SUM, AVG, MIN/MAX). WHERE filters rows, HAVING filters groups.
- NULL and three-valued logic: why NULL = NULL is not true, why COUNT(col) != COUNT(*), why NULLs silently vanish from averages.
- CASE logic in SQL as if-else statements
- Wrong-query gallery I: plausible-looking single-SELECT bugs throughout that students fix with callouts.
- At the end near the conclusion have a table that shows SQL commands and their Python Pandas equivalents.
- Appendix: Pointer to practice sites (DataLemur, StrataScratch, LeetCode SQL)

Goal: student can write multi-table joins and aggregations in a single SELECT, predict how a join changes row counts, explain NULL's silent effects on aggregates, use CASE for conditional logic and spot errors.

Notebook 3: Advanced SQL on Real Data

- The naming ladder: subqueries (an unnamed query inside another) -> CTEs with WITH (name it for one query; readable, stackable) -> views with CREATE VIEW. A CTE is scoped to one query, a view persists in database file.
- Window functions lite: running totals, ranks, lag/lead OVER (PARTITION BY ... ORDER BY ...) as "GROUP BY that keeps your rows."
- INSERT / UPDATE / DELETE: databases are living objects, not archives
- How to run SQL safely in Python. We show the f-string version, and show a SQL-Injection. Rule: SQL and data travel separately
- Wrong-query gallery II: plausible-looking more advanced bugs throughout that students fix with callouts.
- Conclude with the pandas translation table, part 2 (transform for windows, assign chains for CTEs) attached with part 1 into one downloadable cheatsheet.
- Appendix (optional): Take the example further; mirror econometrics workflows in SQL: build a regression-ready panel (join wages to demographics to region, aggregate to person-year).

Goal: student can compose readable multi-step queries with subqueries, CTEs, and views; use simple window functions; modify data with DML; run SQL safely from Python, explain injection and spot errors.

Notebook 4: Cleaning Real Data

- Entity resolution: the same respondent appears as "Smith, J.", "John Smith", and "J. Smith ". Blocking, exact match on cleaned keys, then fuzzy matching (string distance) as the escalation path.
- String and date wrangling in SQL: TRIM, UPPER/LOWER, CAST, and the classic traps: '1,234' stored as text, three date formats in one column, strftime to normalize. CASE-based recoding of inconsistent categoricals ('BC', 'B.C.', 'British Columbia' all are one value).
- wide vs long in SQL (pivot/unpivot), and when each shape is right.
- Outliers: find the $9,999,999 wage entry (sentinel value, not a billionaire). Simple robust rules (percentile fences, median-based flags), flag the outliers/document don't outright delete
- Imputation: NULL vs 0 vs mean-fill vs leave-it, and how each choice changes the regression a will run on this data later (NB7). One concrete demo of mean-imputation biasing a variance estimate.
- Cleaning Log: every fix as a reproducible SQL script, never hand-edits to the raw data. Raw data is immutable; cleaning is code.

Goal: student can resolve duplicate entities with exact and fuzzy matching, clean strings/dates/categoricals in SQL, reshape between wide and long, flag outliers and justify an imputation choice, and explain why cleaning must be reproducible code applied to immutable raw data.

Notebook 5: Designing Data: Modeling and Schemas.

- Entity Relationship (ER) modeling lite: entities, attributes, relationships, cardinality (1-1, 1-N, M-N)
- Keys and integrity: primary keys, foreign keys, referential integrity; write the Data Definition Language (DDL) (CREATE TABLE, constraints, CHECK); then try to break it by inserting an orphaned response, a duplicate respondent and see how the database refuses.
- Connect all of this to tidy data (Wickham)
- Grain of a data table and how it affects what an econometric/ML model can find.
- Database normalization and when to denormalize -> bridge to NB7.

Goal: student can draw an ER diagram for a real research project, translate it to DDL with keys and constraints, explain referential integrity, "one fact, one place" and database normalization.

Notebook 6: Storage, Speed, and Transactions

- ACID (atomicity, consistency, isolation, durability) and Transactions. Run BEGIN, insert half of the data, crash on purpose, ROLLBACK, show nothing persisted. What “must never lose a row" really means.
- Indexes: (hash map, B-Tree): First we run a slow lookup, then CREATE INDEX, run it again. Phone-book example for why it works, plus the trade-off (writes slow down). Include EXPLAIN QUERY PLAN for high level overview of the query's execution strategy.
- Row vs. column storage: benchmark the same query on the same data in SQLite (rows) vs DuckDB (columns) and see the order-of-magnitude gap; explain why (scan only the columns you need, compression).
- File formats as storage choices: CSV, Parquet, .json etc. Benchmark file size and read speed

Goal: student can run and roll back a transaction, explain ACID, speed up a query with an index and prove it via the query plan, explain row vs. column storage with benchmark evidence, and justify Parquet over CSV.

Notebook 7: Two Different Kinds of Databases

- OLTP vs OLAP from first principles: a university registration system (many tiny writes, must never lose a row) vs an enrollment-trends analysis (few huge reads). One database can't be optimal for both. Show and justify the modern split.
- The warehouse pattern / Denormalization: star schema (fact + dimension tables) built from our NB5 database, fit it to an OLS and see the coefficients (Set-up NB9).
- ETL (Extract, Transform, Load) as the bridge from the transactional schema to the analytical one.
- Warehouses vs Data Lakes: warehouses are curated star schemas, and a data lake is a Parquet files in cheap storage, and the feature-store/training-data connection to ML systems.
- Still needs an in-code example here (TODO)
- Appendix A: Modern Database landscape. Just an info dump talking about modern tools: SQLite, DuckDB, PostgreSQL, MySQL/MariaDB, SQL Server. Cloud warehouse: BigQuery, RedShift, Databricks, Snowflake
- Appendix B (optional): Remote connecting to a database: connection strings, credentials in environment variables (never hardcoded), and warehouse etiquette

Goal: student can explain OLTP vs. OLAP and justify the split with evidence from NB6, build a small star schema from a normalized source, write the ETL that populates it, and describe how warehouses and lakes feed ML.

Notebook 8: Pipelines I – Scripts to Graphs

- We run a database update script (ETL NB7) (download, clean, load, aggregate, top to bottom). The first run is successful and on the second we force a timeout and kill it on purpose, then re-run it. Duplicated rows, double-counted aggregates, a wrong OLS coefficient at the end.
- idempotency (re-run safely: DELETE-then-INSERT / INSERT OR REPLACE)
- incremental loads (only fetch what's new)
- logging (what did run?) retries as a wrapper function.
- Dependency: Draw the script as boxes and arrows; realize the arrows are the real program and the top-to-bottom ordering was an accident.
- Formalize: a pipeline is a directed acyclic graph. Build it in networkx: tasks as nodes, dependencies as edges. Define “acyclic.” Students have seen it this whole time, the “You are here” for the stream has been a DAG all along. Connect our strong prAxIs network analysis materials.
- Students are given a short executor and their job is to read it, predict its behavior, and sabotage it (add a cycle, watch it refuse; fail a middle task, observe which downstream tasks are skipped and which independent branches still run).
- Show some wrong pipelines and students identify and fix issues (like with SQL commands in earlier notebooks).
- Finally, students write their own DAG from given nodes + edge list.

Goal: student can explain idempotency, incremental loads, and retries via failures they caused; represent a workflow as a networkx DAG and justify why acyclicity matters; predict what a graph executor will run, skip, and refuse.

Notebook 9: Pipelines II – Quality, Lineage and Time

- We start again with a failure, building off of NB8. A new month of the data arrives subtly broken (a column renamed, wages in cents instead of dollars). The NB8 pipeline runs green with no errors. The pipeline ran != Data is right.
- Data quality as assertions: write validation checks in SQL against the warehouse (null rate, value ranges, row-count delta vs last month, schema match). Add a validate node in the DAG that stops the pipeline. Similar to the cleaning from NB4 (validation vs known mess cleanup), but in the pipeline.
- Lineage: Pipeline = Directed Graph, "what does this table depend on?" is nx.ancestors() and "what did this bad file poison?" is nx.descendants(). Exercise given original corruption at start of notebook, compute the blast radius and decide what must be recomputed, then recompute only the corrupted part using NB8 executor logic. Provenance as a necessary property of data.
- Time enters the graph: a scheduler is just a loop + a clock + state ("which (task, month) pairs have already succeeded?"). Build and explain a simple/short scheduler.
- Explain and run a backfill.
- Invoke pipeline with one call.
- Link this lesson to industry tools like dbt (Data Build Tool) and Apache Airflow, we build the concepts these tools sell.

Goal: student can write data-quality assertions that halt a pipeline, use graph traversal to reason about lineage and blast radius, explain scheduling and backfills as state over time, and articulate why "succeeded" != "correct."

Notebook 10: Databases for AI: Embeddings & Vector Search & Different Data

- One nested JSON API response queried with DuckDB/SQLite JSON functions. Shows that data storage follows data shapes. Warmup exercise and intro.
- Our data warehouse now has a text column. Something like sentiment or survey comments. Task: "find responses about job insecurity." Students attack it with NB2–4 tools and it fails, synonyms, phrasing, negation. They can't do it.
- Embeddings as coordinates of meaning. Link to prAxIs NLP material. Cosine similarity between hand-picked sentence pairs, a 2-D projection plot where clusters are visible.
- Vector search in SQL `ORDER BY distance LIMIT k`. The "job insecurity" query now works compare its results against the LIKE query side by side, including where semantic search is worse (exact codes, names, values).
- What "a database attached to a model" means: retrieval-augmented generation (RAG) demystified as (embed question -> similarity search -> stuff results into context).
- Close the whole stream arc with a diagram and summary of what we did: Messy CSV (NB1) -> SQL basic + advanced and cleaning (NB 2-4) -> schema (NB5) -> warehouse (NB7) -> automated, validated pipeline (NB8–9) -> semantically queryable knowledge base (NB10)
- Appendix (like NB7): NoSQL Landscape: MongoDB, Redis, Neo4j, Pinecone/pgvector/Chroma. Info dump of the landscape of NoSQL.

Goal: student can explain why exact-match querying fails on meaning, generate and geometrically interpret embeddings, implement similarity search as an ORDER BY on the existing warehouse, choose between lookup and semantic search for a given question, and describe RAG as a database pattern.

Notebook 11: Capstone: Your Own Data Product

- Students pick any public data source (StatCan, FRED, Our World in Data, an API) and ship the full stack. Raw data -> cleaning -> Schema -> Warehouse -> DAG + Quality Checks -> into one of three possible outcomes for them. a) Semantic-search interface, (b) plotting/report skeleton we provide, (c) auto-retraining model pipeline that connects to the Causal ML stream (pipeline refreshes data -> EconML model retrains -> output report regenerates).
- The deliverable is judged on one question: can a stranger clone it, run one command, and get your result? A short required README with the system's DAG diagram and a lineage statement.
    - Version control and git appear here, but more conceptually this is not a git guide.
- This notebook is more abstracted than the rest as it is meant for students to take their own data/ideas.

Goal: student ships a complete, verifiable data system and can defend every number in the output by tracing its lineage to raw data.

Sources / Useful things to base it on:

- https://www.students.cs.ubc.ca/~cs-368/ (CPSC 368 learning outcomes — coverage benchmark)
- https://duckdb.org/docs/ and https://duckdb.org/docs/current/guides/python/jupyter (DuckDB in Jupyter)
- https://jupysql.ploomber.io/ (SQL magic cells in notebooks)
- https://www.sqlite.org/ and https://github.com/asg017/sqlite-vec (vector search in SQLite)
- https://duckdb.org/docs/stable/core_extensions/vss (DuckDB vector similarity search extension)
- https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/debug.html (dag.test() — runs DAGs in-notebook, no infra)
- https://www.astronomer.io/docs/learn/ (best free Airflow pedagogy on the web)
- Kleppmann, *Designing Data-Intensive Applications* (instructor background; the "why" behind everything)
- https://github.com/DataTalksClub/data-engineering-zoomcamp (reference open curriculum)
- https://r4ds.hadley.nz/data-tidy and Wickham, "Tidy Data" (JSS 2014) (tidy data ↔ normalization bridge, COMET-familiar)
- https://pandera.readthedocs.io/ (lightweight data-quality checks)
- https://www.pywhy.org/EconML/ + Praxis Causal ML stream (the model layer these pipelines feed)
- Sculley et al., "Hidden Technical Debt in Machine Learning Systems" (NeurIPS 2015 — ML code is a small box inside a much larger data-infrastructure system; the canonical citation for "most of an ML system is plumbing")
- StatCan Web Data Service (https://www.statcan.gc.ca/en/developers) and FRED API (pipeline data sources)
- https://data101.org/sp25/syllabus/ Data101 Berkley
- https://cs50.harvard.edu/sql/notes/0/ Harvard CS50 Intro to Databases with SQL
- https://tyler.caraza-harter.com/cs544/f23/syllabus.html UWMadison into to big data systems
- https://www.lse.ac.uk/study-at-lse/summer-schools/summer-school/courses/research-methods/me204 LSE Summer School ME204 "Data Engineering Principles for the Social Sciences"
- https://datascience.julianhinz.com/ Data Science for Economists
- https://www.oreilly.com/library/view/fundamentals-of-data/9781098108298/